

Memoria Tecnica Del Equipo

28 SEPTIEMBRE

Keyther Davis Tavera Peralta 100765924

Cesar Alonso Sierra Cuevas 100708687

Randy José Sánchez Mota 100552665

Diana ventura



Nombre del
logotipo

1. Descripción del problema y requerimientos principales

Descripción del problema

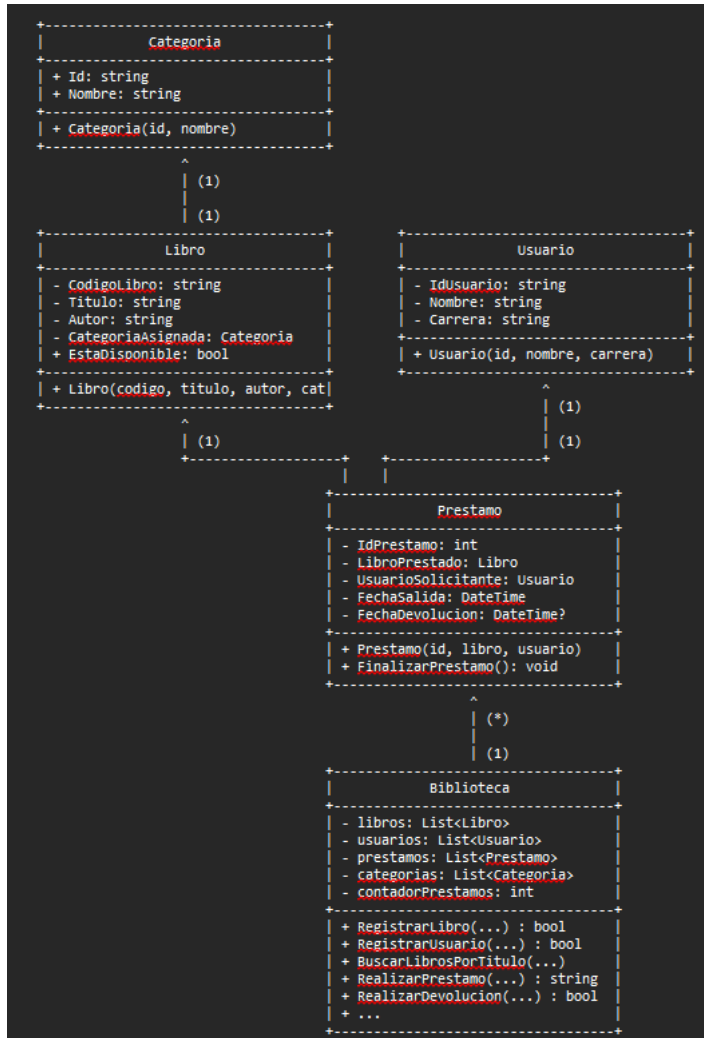
La biblioteca universitaria INF-512 requiere un sistema automatizado para gestionar eficientemente su catálogo de libros, el registro de estudiantes/usuarios y el control de préstamos y devoluciones. El sistema resuelve problemas de descontrol de inventario, duplicidad de registros y falta de visibilidad del estado de disponibilidad de los textos en tiempo real.

Requerimientos principales

- **Gestión de Catálogo:** Registrar libros asociándolos a una categoría válida y controlando que no existan códigos duplicados.
- **Gestión de Usuarios:** Registrar alumnos/usuarios universitarios garantizando identificadores únicos (matrícula/ID).
- **Búsqueda y Consulta:** Permitir la búsqueda de libros por coincidencias de título sin distinguir entre mayúsculas y minúsculas.
- **Control de Préstamos:** Registrar salidas de libros validando la existencia previa del usuario y la disponibilidad del ejemplar.
- **Control de Devoluciones:** Finalizar préstamos activos liberando el estado de disponibilidad del libro correspondiente.
- **Reportes y Consultas:** Consultar los préstamos vigentes por usuario y visualizar un reporte con el inventario completo.

2. Diagrama UML de la implementación final

El siguiente diagrama de clases refleja la estructura estática del sistema, sus atributos, métodos y los tipos de relaciones presentes:



3. Justificación del contenedor seleccionado

Para la gestión de las colecciones internas (libros, usuarios, prestamos, categorias), se eligió List<T> (System.Collections.Generic.List<T>).

Razones de la elección:

1. Dinamicidad de tamaño: Las bibliotecas operan en entornos donde el volumen de datos cambia constantemente. List<T> ajusta automáticamente su capacidad conforme se añaden o eliminan elementos.

2. **Compatibilidad directa con LINQ:** `List<T>` implementa `IEnumerable<T>`, lo que permite ejecutar operaciones avanzadas como `FirstOrDefault`, `Where` y `Any` con un rendimiento óptimo y una sintaxis limpia.
3. **Acceso indexado e iteración veloz:** Al estar internamente respaldada por un arreglo continuo en memoria, proporciona lecturas rápidas e iteraciones eficientes para la generación de reportes.

Criterio / Propiedad	Arreglo Tradicional (<code>T[]</code>)	Lista Genérica (<code>List<T></code>)
Tamaño/Capacidad	Fijo. Definido durante la instanciación.	Dinámico. Crece y decrece automáticamente.
Rendimiento de inserción	$O(1)$ si se conoce el índice; redimensionar requiere manual copia $O(n)$.	$O(1)$ amortizado al agregar con <code>.Add()</code> .
Seguridad de tipos	Estático pero rígido.	Alta seguridad gracias a los genéricos (<code>List<T></code>).
Uso de Memoria	Mínimo y contiguo. Sin sobrecosto de gestión.	Ligeramente mayor debido al buffer de reserva interna.
Métodos y Flexibilidad	Limitado (métodos estáticos de <code>System.Array</code>).	Amplio (<code>Add</code> , <code>Remove</code> , <code>Find</code> , <code>Where</code> , <code>Count</code>).
Idoneidad en este proyecto	Deficiente (se desconoce cuántos libros se registrarán).	Ideal (permite inserciones continuas reasignar memoria manualmente).

5. Análisis de al menos tres relaciones entre clases

1. Asociación Directa: Libro $\rightarrow$ Categoría (Multiplicidad 1:1)

- **Descripción:** Un libro pertenece a una única categoría.
- **Implementación:** La clase Libro almacena una referencia en la propiedad CategoríaAsignada. Si la categoría se elimina del sistema, la categoría conceptual sigue existiendo, evidenciando una relación de asociación donde Libro consulta la información del objeto Categoría.

2. Agregación: Biblioteca $\rightarrow$ Libro (Multiplicidad 1:N)

- **Descripción:** La biblioteca agrupa un conjunto de libros.
- **Implementación:** Biblioteca administra la lista List<Libro>. Es una agregación porque el ciclo de vida del objeto Libro no depende exclusivamente del contenedor: si la biblioteca destruye la lista o renueva la instancia, la abstracción del objeto libro puede mantenerse en reportes o subsistemas externos.

3. Asociación Temporal / Transaccional: Prestamo $\rightarrow$ Libro y Usuario (Multiplicidad 1:1 cada una)

- **Descripción:** Un registro de préstamo conecta de forma explícita un Usuario con un Libro en un instante determinado.
- **Implementación:** La clase Prestamo mantiene referencias directas a LibroPrestado y UsuarioSolicitante. Esta relación vincula dos entidades independientes a través de una entidad transaccional sin modificar la estructura interna de ninguna de las dos.

6. Análisis breve de cohesión, acoplamiento, modularidad y distribución de responsabilidades

- **Cohesión:** Alta. Cada clase está focalizada en un único dominio funcional. Libro, Usuario y Categoría operan únicamente como contenedores de datos de entidad; Prestamo gestiona el estado y ciclo de vida de la transacción; Biblioteca concentra la lógica del negocio; y Program maneja exclusivamente la entrada/salida de la consola.

- **Acoplamiento: Bajo a Moderado.** La clase de interfaz (Program) depende únicamente de la clase de servicio/lógica (Biblioteca). No interactúa directamente con los detalles de las listas internas ni manipula directamente las reglas de los préstamos.
- **Modularidad: Bien definida.** El código está organizado en bloques lógicos claros mediante comentarios formales y nombres de espacio identificables (SistemaBiblioteca). Las capas de presentación, lógica y datos están desacopladas funcionalmente.
- **Distribución de Responsabilidades (GRASP/SOLID):** Se respeta el principio de Responsabilidad Única (SRP). Si las reglas de UI cambian, solo se modifica Program; si cambian las políticas de préstamos o búsquedas, solo se modifica Biblioteca.

7. Capturas representativas de la ejecución y validaciones

```
=====
      SISTEMA DE GESTIÓN DE BIBLIOTECA - INF-512
=====
1. Registrar un Libro
2. Registrar un Usuario
3. Buscar un Libro (por Título)
4. Registrar un Préstamo
5. Registrar una Devolución
6. Consultar Préstamos de un Usuario
7. Ver Reporte de Inventario Completo
8. Salir
-----
Seleccione una opción:
```

```
1. Registrar un Libro
2. Registrar un Usuario
--- REGISTRAR NUEVO LIBRO ---
Ingrese Código del Libro: 1234
Ingrese Título: mega1
Ingrese Autor: k

Categorías disponibles:
- [CAT01] Ingeniería de Software
- [CAT02] Matemáticas
- [CAT03] Base de Datos
Seleccione el ID de la Categoría: cat01

¡Libro registrado exitosamente!

Presione cualquier tecla para continuar...
█
```

```
1. Registrar un Libro
2. Registrar un Usuario
--- REGISTRAR NUEVO USUARIO ---
Ingrese ID Universitario (Matrícula): 100
Ingrese Nombre completo: key
Ingrese Carrera: informatica

¡Usuario registrado exitosamente!

Presione cualquier tecla para continuar...
█
```

```
1. Registrar un Libro
2. Registrar un Usuario
--- BUSCAR LIBRO ---
Ingrese el título o coincidencia a buscar: mega

Resultados encontrados (2):
- Código: 123 | mega de key [Ingeniería de Software] -> Estado:
Disponible
- Código: 1234 | mega1 de k [Ingeniería de Software] -> Estado:
Disponible

Presione cualquier tecla para continuar...
█
```

```
--- PROCESAR PRÉSTAMO ---
Ingrese ID del Usuario: 1005
Ingrese Código del Libro: 12345

Éxito: Préstamo registrado correctamente.

Presione cualquier tecla para continuar...
█
```

--- PROCESAR DEVOLUCIÓN ---

Ingrese el Código del libro que se va a devolver: 12345

¡Devolución registrada con éxito! El libro vuelve a estar disponible.

Presione cualquier tecla para continuar...

--- CONSULTAR PRÉSTAMOS ACTIVOS DE UN USUARIO ---

Ingrese el ID del usuario: 1005

El usuario no tiene préstamos activos pendientes en este momento.

Presione cualquier tecla para continuar...

REPORTE DE INVENTARIO COMPLETO

Código: 111	Título: Clean Code	Categoría: Ingeniería de Software
	Estado: Disponible	
Código: 222	Título: C# 12 in a Nutshell	Categoría: Ingeniería de Software
	Estado: Disponible	
Código: 333	Título: Cálculo de una Variable	Categoría: Matemáticas
	Estado: Disponible	
Código: 12345	Título: megaman	Categoría: Ingeniería de Software
	Estado: Disponible	

Presione cualquier tecla para continuar...

SISTEMA DE GESTIÓN DE BIBLIOTECA - INF-512

1. Registrar un Libro
2. Registrar un Usuario
3. Buscar un Libro (por Título)
4. Registrar un Préstamo
5. Registrar una Devolución
6. Consultar Préstamos de un Usuario
7. Ver Reporte de Inventario Completo
8. Salir

Seleccione una opción: 8

PS C:\Users\keyth\OneDrive\Desktop\visual studio\grupo1>

9. Conclusiones sobre las decisiones técnicas adoptadas

1. **Separación entre Lógica e Interfaz:** Mantener la clase Biblioteca independiente de Console.WriteLine garantiza que la lógica del sistema pueda reutilizarse en una API Web, una aplicación de escritorio (WPF/WinForms) o una solución móvil sin requerir cambios estructurales.
2. **Eficiencia en Búsqueda y Filtrado:** La adopción de LINQ combinada con delegados y comparaciones que ignoran mayúsculas/minúsculas simplificó el desarrollo de funciones como BuscarLibrosPorTitulo y ConsultarPrestamosActivosUsuario, manteniendo el código legible y expresivo.
3. **Manejo de Estados con Nulos (DateTime?):** La utilización de tipos anulables para la propiedad FechaDevolucion en la clase Prestamo ofreció una estrategia limpia y representativa para discernir entre transacciones activas e históricas sin necesidad de agregar campos bandera redundantes.